Activity 3.1 Decision Tree

Objective(s):

This activity aims to perform classification using Decision Tree

Intended Learning Outcomes (ILOs):

- Demonstrate how to build the model using Decision Tree.
- Demonstrate how to evaluate the performance of the model.
- Demonstrate how to create tree visualization of the model.

Resources:

- Jupyter Notebook
- kyphosis

Procedure:

Import the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Load the data and check the content of the dataframe using Pandas

```
df = pd.read_csv('kyphosis.csv')
df.head()
```

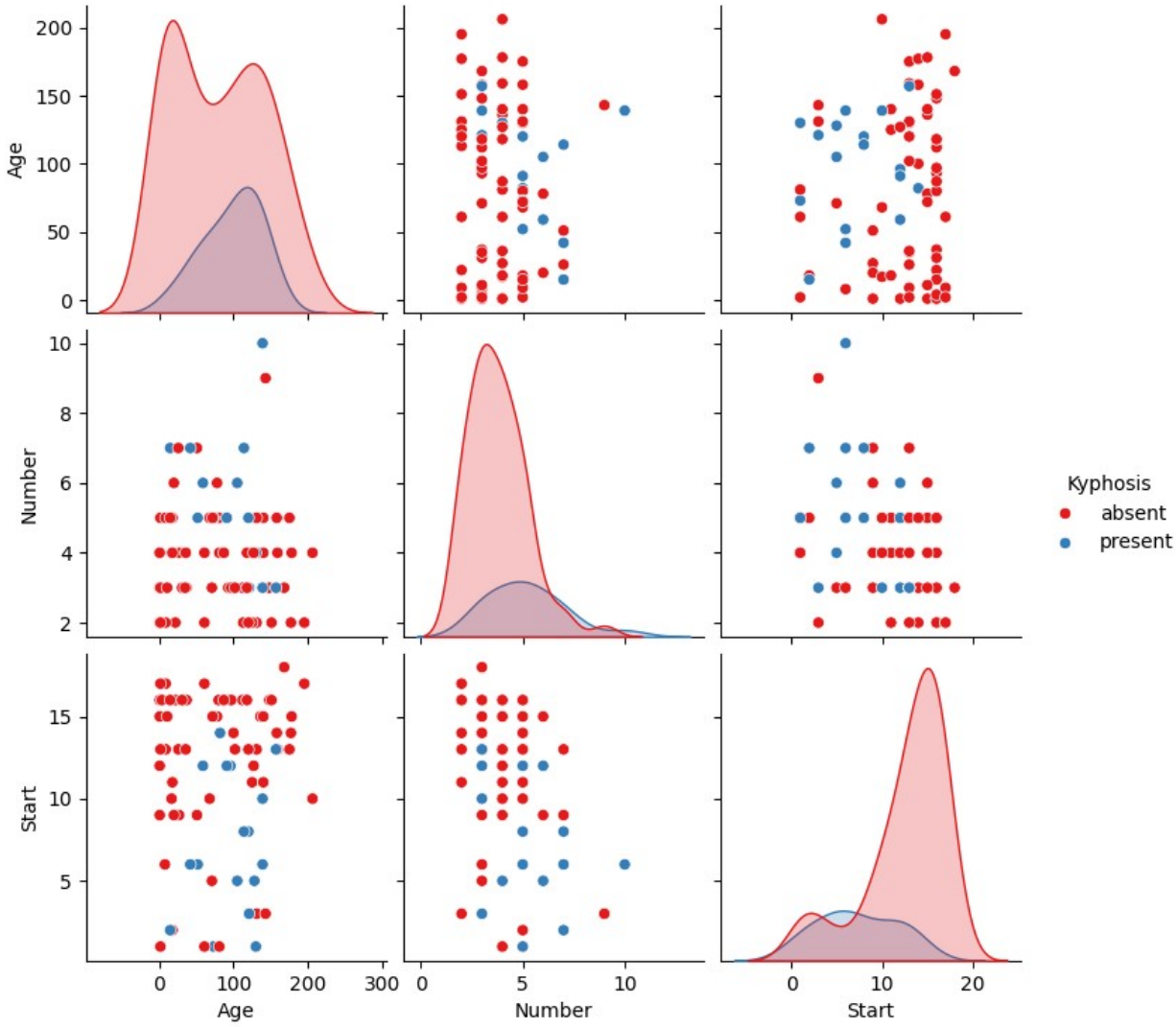

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 81,\n  \"fields\": [\n
{\n      \"column\": \"Kyphosis\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"present\",\n          \"absent\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 58,\n        \"min\": 1,\n        \"max\": 206,\n
\"num_unique_values\": 64,\n        \"samples\": [\n          17,\n
11\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Number\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1,\n        \"min\": 2,\n        \"max\": 10,\n
\"num_unique_values\": 8,\n        \"samples\": [\n          4,\n
7\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

```
\"Start\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 4,\n        \"min\": 1,\n        \"max\": 18,\n
\"num_unique_values\": 16,\n        \"samples\": [\n          5,\n
14\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


Check the pairplot of the data

```
sns.pairplot(df,hue='Kyphosis',palette='Set1')
```

```
<seaborn.axisgrid.PairGrid at 0x7b906549b4d0>
```



Interpret the result of the pairplot

The pariplot above shows that a varied distribution in ages, having data points ranging from 100-200. The number, ranging from 0-5, and start, ranging from 10-20, showed a concentrated distribution. The dataset mostly include data points of people with absent kyphosis.

Build the model using Decision Tree

Split the data into training and test set

```
from sklearn.model_selection import train_test_split

X = df.drop('Kyphosis',axis=1)
y = df['Kyphosis']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30)
```

Train a single decision tree

```
from sklearn.tree import DecisionTreeClassifier

dtree = DecisionTreeClassifier()
dtree.fit(X_train,y_train)
```

```
DecisionTreeClassifier()
```

Determine how many nodes are present and what the depth of this (very large) tree is.

The number of nodes and the maximum actual depth.

```
dtree.tree_.node_count, dtree.tree_.max_depth
```

```
(25, 7)
```

Predict and evaluate the model

```
predictions = dtree.predict(X_test)

from sklearn.metrics import
classification_report,confusion_matrix,accuracy_score

#evaluate the performance using classification report
print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

      absent       0.79      0.83      0.81        18
     present       0.50      0.43      0.46         7

    accuracy                           0.72        25
   macro avg       0.64      0.63      0.64        25
```

```
weighted avg       0.71      0.72      0.71        25
```

Interpret the precision, recall and f1-score

A high precision and f1-score in classifying absent kyphosis and a low precision and f1-score in classifying present kyphosis was seen. Their recalls of 0.77 and 0.67 were also seen and are slightly close with each other. This led to an overall accuracy of 0.76. It has great accuracy, however, it may not perform well in classifying present kyphosis. This is merely due to data imbalance present in the dataset.

```
#evaluate the performance using confusion matrix
print(confusion_matrix(y_test,predictions))
```

```
[[15  3]
 [ 4  3]]
```

Interpret the performance using classification matrix

The classification matrix presented that the model can greatly classify data points, having 17 true positive predictions and only 5 wrongly predicted points.

```
#evaluate the performance using accuracy score
print(accuracy_score(y_test, predictions))
```

```
0.72
```

Interpret the performance of the model using accuracy score

The accuracy score infers a slightly great performance for the model prediction.

Create a tree visualization of the model

```
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydot

features = list(df.columns[1:])
features
```

```
['Age', 'Number', 'Start']
```

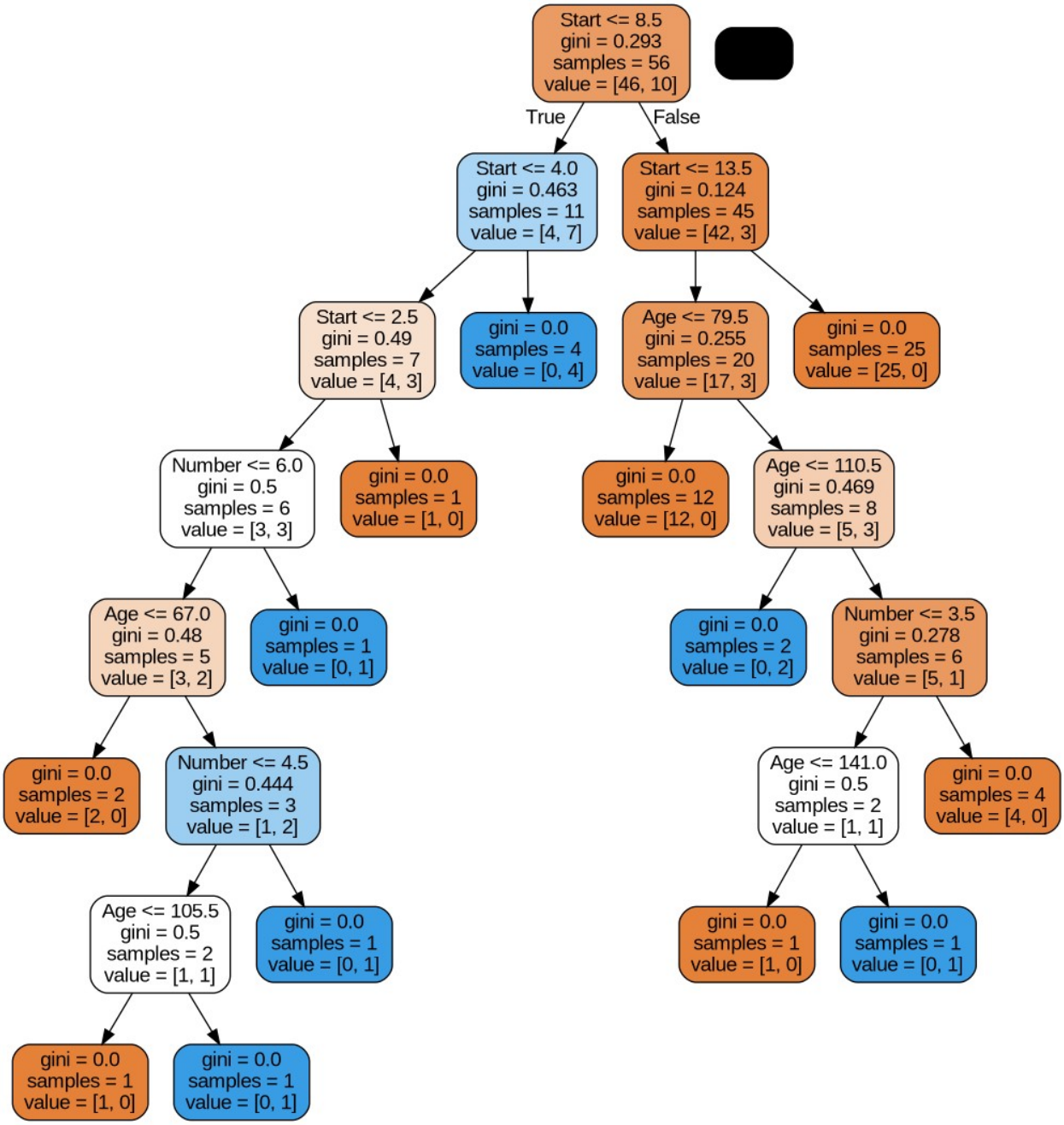
```
from io import StringIO
from IPython.display import Image, display
import pydotplus

from sklearn.tree import export_graphviz
dot_data = StringIO()

export_graphviz(dtree, out_file=dot_data,
```

```
feature_names=features,filled=True,rounded=True)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

    # View the tree image
filename = 'kyphosis .png'
graph.write_png(filename)
img = Image(filename=filename)
display(img)
```



Supplementary Activity:

- Choose your own dataset
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values(NaN), replacing missing data .
- Perform descriptive statistics such as mean, median and mode
- Perform data visualization
- Build the model using Decision Tree
- Evaluate the model using classification report, accuracy and confusion matrix
- Create a tree visualization of the model

Supplementary Activity

Dataset: Penguin Species Dataset

Importing dataset

```
# Importing dataset
df = pd.read_csv("penguins.csv")
df.head(5)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 1000,\n  \"fields\":
[\n    {\n      \"column\": \"species\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"Adelie\",\n          \"Gentoo\",\n
\"Chinstrap\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"island\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Torgersen\",\n          \"Biscoe\",\n
\"Dream\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"culmen_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5.309985167434392,\n        \"min\":
32.4,\n        \"max\": 56.0,\n        \"num_unique_values\": 206,\n
\"samples\": [\n          37.7,\n          50.5,\n          36.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7790570332759408,\n        \"min\": 13.0,\n        \"max\": 21.4,\n
\"num_unique_values\": 80,\n        \"samples\": [\n          13.8,\n
18.4,\n          19.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"flipper_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 13.17166038094355,\n        \"min\":
175.0,\n        \"max\": 235.0,\n        \"num_unique_values\": 58,\n
\"samples\": [\n          195.0,\n          189.0,\n          180.0\n
```

```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
705.3347935976439,\n        \"min\": 2830.0,\n        \"max\":
6317.0,\n        \"num_unique_values\": 786,\n        \"samples\": [\n
3714.0,\n          3352.0,\n          3454.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"Male\",\n          \"Female\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```

```
# Data points
print(f"Data Points: {df.shape[0]}\nColumns: {df.shape[1]}")
```

```
Data Points: 1000
Columns: 7
```

```
# Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            1000 non-null   object
 1   island             1000 non-null   object
 2   culmen_length_mm   990 non-null    float64
 3   culmen_depth_mm    990 non-null    float64
 4   flipper_length_mm  990 non-null    float64
 5   body_mass_g        990 non-null    float64
 6   sex                990 non-null    object
dtypes: float64(4), object(3)
memory usage: 54.8+ KB
```

```
# Data types
df.dtypes
```

```
species               object
island                object
culmen_length_mm     float64
culmen_depth_mm      float64
flipper_length_mm    float64
body_mass_g          float64
sex                   object
dtype: object
```

```
# Describe
df.describe()
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"culmen_length_mm\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 336.7891037609659,\n
\"min\": 5.309985167434392,\n        \"max\": 990.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
43.817777777777785,\n          43.8,\n          990.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
344.72465376316666,\n        \"min\": 1.7790570332759408,\n
\"max\": 990.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          17.335555555555555,\n          17.8,\n
990.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"flipper_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 296.3774700965604,\n        \"min\":
13.17166038094355,\n        \"max\": 990.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
199.22424242424242,\n          195.0,\n          990.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1854.287592761864,\n        \"min\": 705.3347935976439,\n
\"max\": 6317.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          4103.311111111111,\n          3882.0,\n
990.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Data Cleaning

```
# Create copy
df_clean = df.copy()

# Clean missing rows
df_clean = df.dropna()

df_clean.shape
```

```
(951, 7)
```

```
# Get numerical columns
dfc_numeric = df_clean.select_dtypes(include='number')

# Mean
dfc_numeric.mean()
```

```
culmen_length_mm       43.825026
culmen_depth_mm        17.324711
flipper_length_mm     199.191377
```

```
body_mass_g          4096.220820
dtype: float64
```

```
# Median
dfc_numeric.median()
```

```
culmen_length_mm       43.9
culmen_depth_mm        17.8
flipper_length_mm     195.0
body_mass_g          3880.0
dtype: float64
```

```
# Mode
dfc_numeric.mode()
```


```
{"summary":"{\n  \"name\": \"dfc_numeric\",\n  \"rows\": 4,\n
\"fields\": [\n    {\n      \"column\": \"culmen_length_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 38.3,\n        \"max\": 38.3,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          38.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 18.2,\n        \"max\": 18.2,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          18.2\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"flipper_length_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 189.0,\n        \"max\": 189.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          189.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
198.1302181226613,\n        \"min\": 3318.0,\n        \"max\":
3755.0,\n        \"num_unique_values\": 4,\n        \"samples\": [\n
3540.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

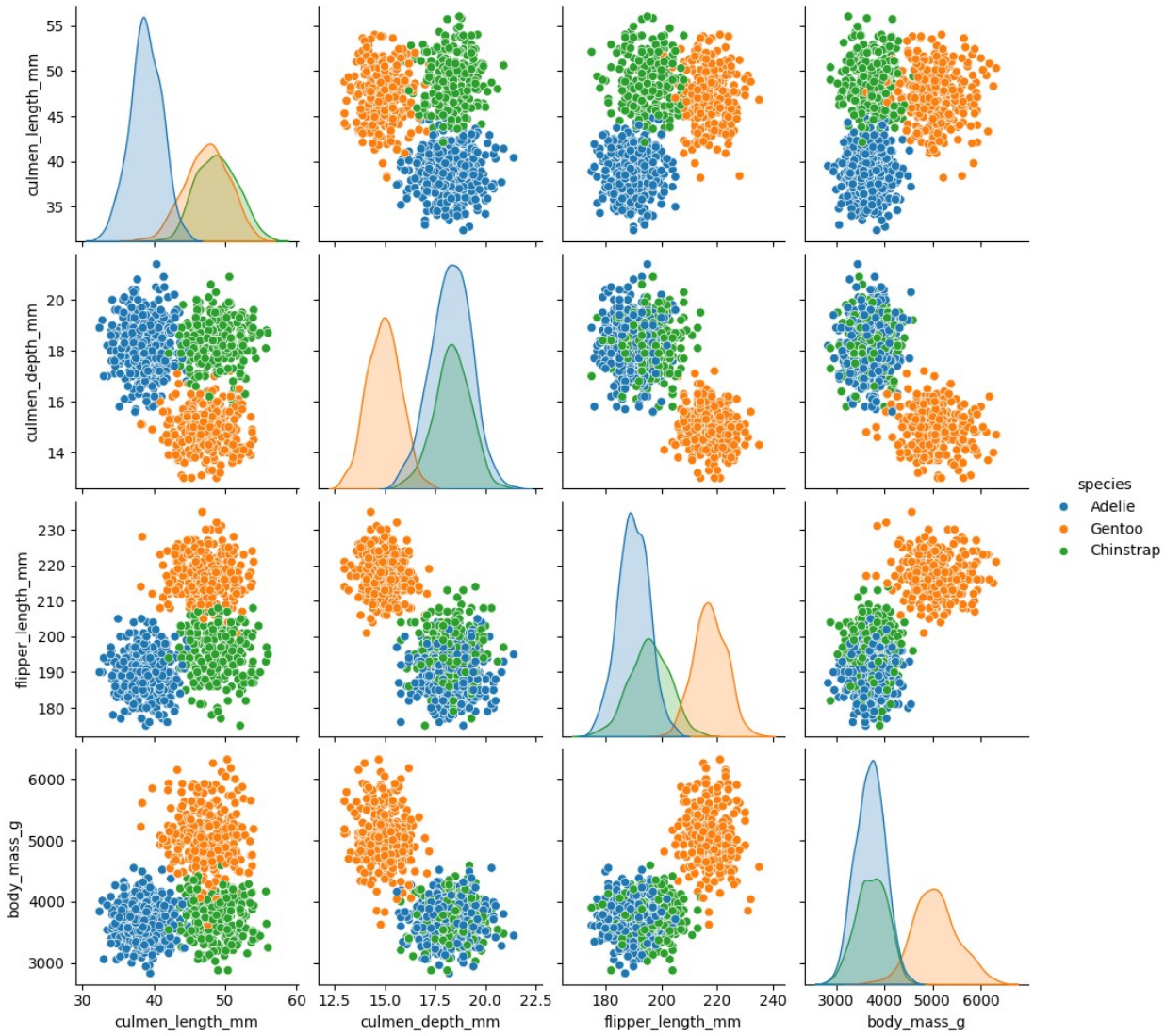

```
# Clean df
df_clean.head()
```


```
{"summary":"{\n  \"name\": \"df_clean\",\n  \"rows\": 951,\n
\"fields\": [\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Adelie\",\n          \"Gentoo\",\n          \"Chinstrap\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"island\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Torgersen\",\n          \"Biscoe\",\n
```

```
\"Dream\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"culmen_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5.326905329930786,\n        \"min\":
32.4,\n        \"max\": 56.0,\n        \"num_unique_values\": 204,\n
\"samples\": [\n          37.7,\n          50.5,\n          52.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7705037331010263,\n        \"min\": 13.0,\n        \"max\": 21.4,\n
\"num_unique_values\": 79,\n        \"samples\": [\n          16.1,\n
18.4,\n          19.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"flipper_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 13.183449324835415,\n        \"min\":
175.0,\n        \"max\": 235.0,\n        \"num_unique_values\": 58,\n
\"samples\": [\n          195.0,\n          189.0,\n          180.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
703.6957802823009,\n        \"min\": 2830.0,\n        \"max\":
6317.0,\n        \"num_unique_values\": 763,\n        \"samples\": [\n
4392.0,\n          4153.0,\n          4231.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"Male\",\n          \"Female\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"df_clean"}
```


Data visualization

```
sns.pairplot(data=df_clean, hue="species")
plt.show()
```



Model Training

```
X = df_clean.select_dtypes(include="number")
y = df_clean["species"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30)

dtree = DecisionTreeClassifier()
dtree.fit(X_train,y_train)

DecisionTreeClassifier()

# Node count, Max depth
dtree.tree_.node_count, dtree.tree_.max_depth

(35, 8)
```

```
# Predict
predictions = dtree.predict(X_test)

# Evaluate the performance using classification report
print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

      Adelie       0.99      0.98      0.99       140
   Chinstrap       0.86      0.99      0.92        73
      Gentoo       1.00      0.88      0.93        73

    accuracy                           0.95       286
   macro avg       0.95      0.95      0.95       286
weighted avg       0.96      0.95      0.96       286
```

```
# Evaluate the performance using confusion matrix
print(confusion_matrix(y_test,predictions))
```

```
[[137   3   0]
 [  1  72   0]
 [  0   9  64]]
```

```
# Evaluate the performance using accuracy score
print(accuracy_score(y_test, predictions))
```

```
0.9545454545454546
```

```
# Get features
features = list(X.columns)
features
```

```
['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm',
'body_mass_g']
```

```
# Create decision tree
dot_data = StringIO()

export_graphviz(dtree, out_file=dot_data, feature_names=features,
filled=True, rounded=True)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

# View the tree image
filename = 'penguins_decisiontree.png'
graph.write_png(filename)
img = Image(filename=filename)
display(img)
```



Conclusion

The activity involves the application of the Decision Tree Classifier, which is presented in classifying in absent and present Kyphosis. To do this, the dataset splitted to 70% by 30% is feeded into the decision tree classifier function, which checks for the value of numerical features of each data point. With this, a slightly accurate model was made, with the model accuracy of 72% and the absent Kyphosis being correctly classified more than present Kyphosis. This is mainly due to the data imbalances found in the dataset, where absent Kyphosis data are dominant than present Kyphosis. The decision tree used in the procedure was also printed, showing its max nodes and depth. The same procedure was made in the supplementary activity, but was used for penguin species classification. Applying the decision tree classifier returned a high model accuracy with 95%, having high precision, recall, and f1 score ranging from 86% to 100%. A decision tree was also created to show the workflow made in classifying testing data. The decision tree was also made to check how the classifier model predicts data.